CS24301 — THEORY (3.0 CR) MODULE II NOTES BIT MESRA NEP SCHEME

Module 2: Syntax Analysis

Context-Free Grammars, LL(1), SLR(1), CLR(1), LALR(1) & Parsing Conflicts

Module II: Syntax Analysis & Parsing Techniques — 10-Topic Master Syllabus Guide

Topic 8: Introduction to Syntax Analysis & Context-Free Grammars (CFG)

Topic 9: Grammar Transformations (Left Recursion Elimination & Left Factoring)

Topic 10: Recursive Top-Down Parsers (Recursive Descent & Backtracking)

Topic 11: Non-Recursive Predictive Parsers (Stack-Driven Parsing Model)

Topic 12: LL(1) Parser Design (Inductive FIRST & FOLLOW Math, Table & Stack Trace)

Topic 13: Bottom-Up Parsers (Shift-Reduce Parsing, Handles & Handle Pruning)

Topic 14: Variants of LR Parsers (LR(0), SLR(1), CLR(1) & LALR(1) Hierarchy)

Topic 15: Parsing Conflict Resolution (Shift-Reduce & Reduce-Reduce Disambiguation)

Topic 16: Detection of Syntax Errors & Error Recovery Strategies (Panic Mode)

Topic 17: Reporting Syntax Errors & Diagnostic Error Productions

Topic 8: Introduction to Syntax Analysis & Context-Free Grammars

Syntax Analysis (Parsing) is the second phase of the compiler pipeline. It takes the stream of atomic tokens emitted by the lexical scanner and verifies whether the sequence conforms to the formal syntactic specification of the source programming language, organizing the tokens into an explicit hierarchical **Parse Tree**.

Context-Free Grammar (CFG) Formal 4-Tuple:

$$G = (V, \Sigma, R, S)$$

- V: Finite set of **Non-Terminal symbols** (Syntactic variables, e.g., $\{E, T, F\}$). - Σ: Finite set of **Terminal symbols** (Token alphabet, e.g., $\{\mathbf{id}, +, *, (,)\}$), where $V \cap \Sigma = \emptyset$. - R: Finite set of **Production Rules** of the form $A \to \alpha$, where $A \in V$ and $\alpha \in (V \cup \Sigma)^*$. - S: Designated **Start Symbol** ($S \in V$).

Ambiguity in Grammars

A grammar G is **Ambiguous** if there exists at least one input string $w \in L(G)$ that admits **two or more distinct Parse Trees** (or equivalently, two distinct Leftmost Derivations). Ambiguity is unacceptable in compilers because it assigns conflicting semantics/precedences to valid programs.

Proof of Ambiguity: Dangling-Else Grammar

$$\text{Grammar: } S \to \textbf{if } E \textbf{ then } S \mid \textbf{if } E \textbf{ then } S \textbf{ else } S \mid a$$

String: `if E1 then if E2 then S1 else S2` produces two valid parse trees depending on which `if` the `else` clause binds to:

```
Tree 1 (Else binds to inner if - Standard C Rule):
        S
       /|\ \
     if E1 then S
              /|\ \ \
            if E2 then S1 else S2

Tree 2 (Else binds to outer if - Grammatically valid without disambiguation):
        S
      / | \ \ \
    if E1 then S else S2
             /|\
           if E2 then S1
```

Topic 9: Grammar Transformations for Top-Down Parsing

9.1 Elimination of Immediate Left Recursion:

A grammar is **Left-Recursive** if it has a non-terminal A such that $A \Rightarrow^{+} A\alpha$. Top-down parsers enter infinite recursion loops on left-recursive grammars.

Left Recursion Elimination Formula:

$$A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$$

Is transformed into an equivalent right-recursive grammar by introducing a new non-terminal A':

$$\mathbf{A \to \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A'}$$

$$\mathbf{A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \epsilon}$$

🏛️ Step-by-Step Solved Problem: Eliminating Left Recursion from Expression Grammar

Original Left-Recursive Grammar:

$$E \to E + T \mid T$$

$$T \to T * F \mid F$$

$$F \to (E) \mid \mathbf{id}$$

Transformed Right-Recursive Grammar:

$$E \to TE', \quad E' \to +TE' \mid \epsilon$$

$$T \to FT', \quad T' \to *FT' \mid \epsilon$$

$$F \to (E) \mid \mathbf{id}$$

9.2 Elimination of Indirect (General) Left Recursion Algorithm

```
Algorithm: Indirect Left Recursion Elimination
1. Arrange non-terminals in some order A1, A2, ... , An.
2. for i = 1 to n do:
     for j = 1 to i - 1 do:
       replace each production Ai → Aj gamma by Ai → delta1 gamma | delta2 gamma | ...
       where Aj → delta1 | delta2 | ... are current Aj-productions.
     eliminate immediate left recursion on Ai-productions.
```

9.3 Left Factoring (Prefix Disambiguation):

When two productions for a non-terminal share a common prefix, a top-down parser cannot determine which branch to choose without looking ahead multiple tokens.

Left Factoring Transformation Formula:

$$A \to \alpha\beta_1 \mid \alpha\beta_2 \mid \gamma \implies \mathbf{A} \to \alpha\mathbf{A'} \mid \gamma, \quad \mathbf{A'} \to \beta_1 \mid \beta_2$$

Topic 10 & 11: Recursive Descent vs. Non-Recursive Predictive Parsers

Feature	Recursive Descent Parsing (Topic 10)	Non-Recursive Predictive LL(1) Parsing (Topic 11)
Implementation	A collection of recursive C/Python functions, one for each non-terminal in the grammar.	An explicit **LIFO Parser Stack** + 2D **Parsing Table** $M[A, a]$ driven by a table-interpreter loop.
Backtracking	May require expensive backtracking if grammar is not LL(1) (exponential time complexity).	Strictly deterministic, zero-backtracking, $O(n)$ linear parsing time.
Memory Overhead	Implicit CPU call stack consumption proportional to maximum derivation tree depth.	Explicit stack storing grammatical symbols + 2D array parsing table $M[A, a]$.

Topic 12: LL(1) Parser Design (FIRST, FOLLOW & Parsing Tables)

Inductive Mathematical Formulations for FIRST and FOLLOW

1. Computation of $\mathrm{FIRST}(\alpha)$ for any string $\alpha \in (V \cup \Sigma)^*$:

- If X is a terminal, $\mathrm{FIRST}(X) = \{X\}$.
- If $X \to \epsilon$ is a production, then $\epsilon \in \mathrm{FIRST}(X)$.
- If X is a non-terminal and $X \to Y_1 Y_2 \dots Y_k$, then add $a \in \mathrm{FIRST}(Y_1)$ to $\mathrm{FIRST}(X)$. If $Y_1 \Rightarrow^* \epsilon$, then add $\mathrm{FIRST}(Y_2)$, and so on. If all $Y_1 \dots Y_k \Rightarrow^* \epsilon$, then add ϵ to $\mathrm{FIRST}(X)$.

2. Computation of $\mathrm{FOLLOW}(A)$ for any non-terminal $A \in V$:

- Add end-marker $\$$ to $\mathrm{FOLLOW}(S)$, where S is the start symbol.
- If there is a production $A \to \alpha B \beta$, then everything in $\mathrm{FIRST}(\beta)$ (except ϵ) is in $\mathrm{FOLLOW}(B)$.
- If there is a production $A \to \alpha B$, or $A \to \alpha B \beta$ where $\mathrm{FIRST}(\beta)$ contains ϵ, then everything in $\mathrm{FOLLOW}(A)$ is in $\mathrm{FOLLOW}(B)$.

🏛️ Step-by-Step Solved Problem: Complete LL(1) Parser Construction

Grammar:

$$E \to TE', \quad E' \to +TE' \mid \epsilon, \quad T \to FT', \quad T' \to *FT' \mid \epsilon, \quad F \to (E) \mid \mathbf{id}$$

Step 1: Computed FIRST & FOLLOW Sets:

Non-Terminal	FIRST Set	FOLLOW Set
E	$\{(, \mathbf{id}\}$	$\{), \$\}$
E'	$\{+, \epsilon\}$	$\{), \$\}$
T	$\{(, \mathbf{id}\}$	$\{+,), \$\}$
T'	$\{*, \epsilon\}$	$\{+,), \$\}$
F	$\{(, \mathbf{id}\}$	$\{+, *,), \$\}$

Step 2: LL(1) Parsing Table $M[A, a]$:

Non-Terminal	$\mathbf{id}$	$+$	$*$	$($	$)$	$\$$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to \mathbf{id}$			$F \to (E)$		

Observation: Every cell $M[A, a]$ has at most ONE entry $\implies$ The grammar is strictly LL(1)!

Step 3: Stack Execution Trace for Input String `id + id * id $`:

Step	Parser Stack	Remaining Input	Action Applied
1	`$ E`	`id + id * id $`	Pop E, Push TE' (from $M[E, \text{id}]$)
2	`$ E' T`	`id + id * id $`	Pop T, Push FT' (from $M[T, \text{id}]$)
3	`$ E' T' F`	`id + id * id $`	Pop F, Push id (from $M[F, \text{id}]$)
4	`$ E' T' id`	`id + id * id $`	**Match `id`**: Pop `id`, advance input
5	`$ E' T'`	`+ id * id $`	Pop T', Push ϵ (from $M[T', +]$)
6	`$ E'`	`+ id * id $`	Pop E', Push $+TE'$ (from $M[E', +]$)
7	`$ E' T +`	`+ id * id $`	**Match `+`**: Pop `+`, advance input
8	`$ E' T`	`id * id $`	Pop T, Push FT' (from $M[T, \text{id}]$)
9	`$ E' T' F`	`id * id $`	Pop F, Push id (from $M[F, \text{id}]$)
10	`$ E' T' id`	`id * id $`	**Match `id`**: Pop `id`, advance input
11	`$ E' T'`	`* id $`	Pop T', Push $*FT'$ (from $M[T', *]$)
12	`$ E' T' F *`	`* id $`	**Match `*`**: Pop `*`, advance input
13	`$ E' T' F`	`id $`	Pop F, Push id (from $M[F, \text{id}]$)
14	`$ E' T' id`	`id $`	**Match `id`**: Pop `id`, advance input
15	`$ E' T'`	`$`	Pop T', Push ϵ (from $M[T', \$]$)
16	`$ E'`	`$`	Pop E', Push ϵ (from $M[E', \$]$)
17	`$`	`$`	**SUCCESSFUL PARSE (ACCEPT)**

Topic 13: Bottom-Up Shift-Reduce Parsing & Handles

A **Bottom-Up Parser** constructs a parse tree for an input string beginning at the leaves (terminals) and working upward towards the root (start symbol). This corresponds to constructing a **Rightmost Derivation in Reverse**.

The Concept of a Handle

A **Handle** of a right-sentential form $\gamma = \alpha\beta w$ is a production rule $A \to \beta$ and a position in γ where β may be replaced by A to produce the previous right-sentential form $\alpha A w$ in a rightmost derivation:

$$S \Rightarrow_{\text{rm}}^{*} \alpha A w \Rightarrow_{\text{rm}} \alpha\beta w$$

Handle Pruning: The process of repeatedly finding the handle β and reducing it to A until the start symbol S is reached.

Topic 14: The LR Parsing Family Hierarchy

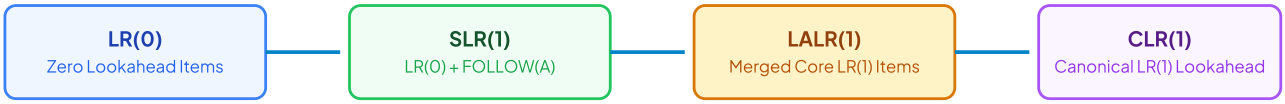

Figure 2.1: The LR Parsing Power Hierarchy ($\text{LR}(0) \subset \text{SLR}(1) \subset \text{LALR}(1) \subset \text{CLR}(1)$)

Parser Variant	Item Representation	Reduction Placement Rule	State Count	Expressive Power
LR(0)	$[A \to \alpha \cdot \beta]$	Reduce action placed in **ALL terminal columns** of state.	K states (Smallest)	Weakest; easily suffers from S-R and R-R conflicts.
SLR(1)	$[A \to \alpha \cdot \beta]$	Reduce action placed **only in columns** $\in \text{FOLLOW}(A)$.	K states (Identical to LR(0))	Significantly more powerful than LR(0); still fails on C-like grammars.
CLR(1)	$[A \to \alpha \cdot \beta,\ a]$ $(a \in \Sigma \cup \{\$\})$	Reduce action placed **only for exact lookahead** a.	Large ($5\times$ to $10\times$ K states)	**Most Powerful** deterministic bottom-up parser.
LALR(1)	Merged $[A \to \alpha \cdot \beta,\ a \mid b]$	Merges CLR(1) states having identical LR(0) cores.	K states (Identical to LR(0))	Industry standard (Yacc / Bison); parses almost all practical programming languages!

14.1 Canonical LR(0) Automaton Construction & Items

An $\text{LR}(0)$ **item** of a grammar G is a production rule with a dot ($\cdot$) placed at some position of the right-hand side. For example, the production $A \to XYZ$ generates four distinct $\text{LR}(0)$ items:

$[A \to \cdot XYZ]$: Indicates we expect to see a string derivable from XYZ.
$[A \to X \cdot YZ]$: Indicates we have recognized X and expect to see YZ.
$[A \to XY \cdot Z]$: Indicates we have recognized XY and expect to see Z.
$[A \to XYZ \cdot]$: Indicates we have recognized the complete handle XYZ and are ready to reduce to A.

Closure and Goto Operational Functions:

$$\text{Closure}(\mathbf{I}) = \mathbf{I} \cup \{[\mathbf{B} \to \cdot\gamma] \mid [\mathbf{A} \to \alpha \cdot \mathbf{B}\beta] \in \text{Closure}(\mathbf{I}),\ \mathbf{B} \to \gamma \in \mathbf{R}\}$$

$$\text{Goto}(\mathbf{I}, \mathbf{X}) = \text{Closure}\big(\{[\mathbf{A} \to \alpha\mathbf{X} \cdot \beta] \mid [\mathbf{A} \to \alpha \cdot \mathbf{X}\beta] \in \mathbf{I}\}\big)$$

Topic 15 – 17: Parsing Conflicts & Error Recovery Strategies

The 2 Major Parsing Conflicts in LR Tables

- **Shift-Reduce (S-R) Conflict:** Parser cannot decide whether to shift the incoming token or reduce the handle currently on top of the stack (e.g., Dangling-Else ambiguity). Resolved by favoring Shift (bind else to innermost if).
- **Reduce-Reduce (R-R) Conflict:** Parser cannot decide which of two distinct production rules $A \to \alpha$ or $B \to \alpha$ to reduce by. Serious design flaw indicating ambiguous grammar.

Syntax Error Recovery Mechanisms:

Panic Mode Recovery: The parser discards incoming tokens until a designated **Synchronizing Token** (such as `;` or `}`) is found in the input stream, pops stack states until an appropriate recovery state is exposed, and resumes parsing.
Phrase-Level Recovery: Replaces a prefix of the remaining input with some string that allows the parser to continue (e.g., inserting a missing semicolon or deleting an extraneous comma).
Error Productions: The grammar is augmented with anticipated common programmer error rules (e.g., $E \to E\ \mathbf{id}\ E$), allowing the parser to construct an error node and emit precise diagnostics.

Topic 17.5: Operator Precedence Parsing

An **Operator Grammar** is a context-free grammar with two restrictions: (1) No production has ϵ on the right-hand side, and (2) No two non-terminals appear adjacent on the right-hand side (e.g., $E \to EE$ is illegal).

The 3 Operator Precedence Relations: - $a \lessdot b$: Terminal a yields precedence to terminal b (Shift b onto stack). - $a \doteq b$: Terminal a has same precedence as b (Part of same operator, e.g., (and)). - $a \gtrdot b$: Terminal a takes precedence over b (Reduce handle bounded by a).

🧠 M2 Active Recall & 10-Question University Exam Master Bank

Q1. Prove that merging CLR(1) states to form LALR(1) can NEVER produce a Shift-Reduce conflict. (8 Marks)

Suppose two CLR(1) states I_1 and I_2 have identical LR(0) cores. If an S-R conflict were to appear in the merged state $I_{1,2}$, there must exist an item $[A \to \alpha \cdot, a]$ and an item $[B \to \beta \cdot a\gamma, b]$.
Since shift actions depend strictly on the presence of the terminal a following the dot in the core $[B \to \beta \cdot a\gamma]$ (which is identical in both I_1 and I_2), the shift action was already present in both individual states.
The lookahead a for $[A \to \alpha \cdot]$ was present in either I_1 or I_2. Thus, the S-R conflict must have already existed in that original CLR(1) state!
Therefore, merging states with identical cores **can never introduce new Shift-Reduce conflicts** (though it can occasionally introduce Reduce-Reduce conflicts).

Q2. Calculate FIRST and FOLLOW sets for the following grammar and construct LL(1) parsing table: (10 Marks)
$S \to aBDh, \quad B \to cC, \quad C \to bC \mid \epsilon, \quad D \to EF, \quad E \to g \mid \epsilon, \quad F \to f \mid \epsilon$
FIRST Sets: $\text{FIRST}(S) = \{a\}, \text{FIRST}(B) = \{c\}, \text{FIRST}(C) = \{b, \epsilon\}, \text{FIRST}(D) = \{g, f, \epsilon\}, \text{FIRST}(E) = \{g, \epsilon\}, \text{FIRST}(F) = \{f, \epsilon\}$.
FOLLOW Sets: $\text{FOLLOW}(S) = \{\$\}, \text{FOLLOW}(B) = \{g, f, h\}, \text{FOLLOW}(C) = \{g, f, h\}, \text{FOLLOW}(D) = \{h\}, \text{FOLLOW}(E) = \{f, h\}, \text{FOLLOW}(F) = \{h\}$.
Table $M[A, a]$ has zero overlapping entries $\implies$ Grammar is LL(1).

Q3. Differentiate between Top-Down and Bottom-Up Parsing across 5 dimensions. (8 Marks)

1. **Starting Point:** Top-Down starts at Root (S); Bottom-Up starts at Leaves (input terminals).
2. **Derivation Strategy:** Top-Down constructs Leftmost Derivation; Bottom-Up constructs Rightmost Derivation in Reverse.
3. **Grammar Restrictions:** Top-Down cannot handle Left Recursion; Bottom-Up handles both Left and Right recursion seamlessly.
4. **Lookahead Power:** LL(1) sees 1 token lookahead; LR(1) can see handles after complete subtrees are built.
5. **Implementation:** Top-Down uses recursive descent or $M[A, a]$ stack; Bottom-Up uses Shift-Reduce DFA state tables.

Q4. What is a Handle in Shift-Reduce parsing? Why is Handle Pruning significant? (6 Marks)

A **Handle** is a substring β in a right-sentential form $\alpha\beta w$ that matches the right-hand side of a production $A \to \beta$ such that replacing β with A represents the reverse of the previous step in a rightmost derivation ($S \Rightarrow_{\text{rm}}^{*} \alpha A w \Rightarrow_{\text{rm}} \alpha\beta w$). **Handle Pruning** guarantees that the bottom-up parser reconstructs the exact valid parse tree from leaves to start symbol without dead-end backtracking.

Q5. Explain the Yacc / Bison LALR(1) Parser Generator Architecture and conflict resolution mechanisms. (8 Marks)

Yacc takes a grammar specification (`parser.y`) and constructs an LALR(1) state table. It resolves Shift-Reduce conflicts using default rules (favor Shift) or explicit `%left`, `%right`, `%nonassoc` precedence directives. It resolves Reduce-Reduce conflicts by reducing via the rule listed earliest in the specification file.

Topic 17.6: Complete LR(0), SLR(1), CLR(1), LALR(1) & Operator Precedence Traces

🏛️ **Step-by-Step Solved Problem: Canonical Collection of LR(0) Items for Grammar $S \to AA, A \to aA \mid b$**

Augmented Grammar: $S' \to S, \quad S \to AA, \quad A \to aA \mid b$

- **State** $I_0 = \text{Closure}(\{[S' \to \cdot S]\})$:

```
[S' → . S]
[S  → . A A]
[A  → . a A]
[A  → . b]
```

- $\text{Goto}(I_0, S) = I_1 : \{[S' \to S\cdot]\} \implies \text{ACCEPT}$
- $\text{Goto}(I_0, A) = I_2 : \{[S \to A \cdot A], [A \to \cdot aA], [A \to \cdot b]\}$
- $\text{Goto}(I_0, a) = I_3 : \{[A \to a \cdot A], [A \to \cdot aA], [A \to \cdot b]\}$
- $\text{Goto}(I_0, b) = I_4 : \{[A \to b\cdot]\} \implies \text{Reduce } A \to b$
- $\text{Goto}(I_2, A) = I_5 : \{[S \to AA\cdot]\} \implies \text{Reduce } S \to AA$
- $\text{Goto}(I_3, A) = I_6 : \{[A \to aA\cdot]\} \implies \text{Reduce } A \to aA$

🏛️ Operator Precedence Parsing Table Construction & Evaluation Trace

Grammar: $E \to E + E \mid E * E \mid \mathbf{id}$

Operator	$+$	$*$	$\mathbf{id}$	$
$+$	$\cdot >$	$< \cdot$	$< \cdot$	$\cdot >$
$*$	$\cdot >$	$\cdot >$	$< \cdot$	$\cdot >$
$\mathbf{id}$	$\cdot >$	$\cdot >$	Error	$\cdot >$
$	$< \cdot$	$< \cdot$	$< \cdot$	Accept

Stack Evaluation Trace for Input `id + id * id $`:

1. Stack: `$< \cdot\mathbf{id}$ | Input: `+ id * id $` $\implies Reduce \mathbf{id}$
2. Stack: `$< \cdot + < \cdot\mathbf{id}$ | Input: `* id $` $\implies Reduce \mathbf{id}$
3. Stack: `$< \cdot + < \cdot * < \cdot\mathbf{id}$ | Input: `$` $\implies Reduce \mathbf{id}$
4. Stack: `$< \cdot + < \cdot * $` | $Input: $` $\implies * \cdot > \$ \implies$ Reduce $*$: Pop $*$
5. Stack: `$< \cdot + $` | $Input: $` $\implies + \cdot > \$ \implies$ Reduce $+$: Pop $+$
6. Stack: `$` | $Input: $` $\implies$ **ACCEPT!**

Q6. Compare LL(1), SLR(1), CLR(1), and LALR(1) parsers across 5 fundamental dimensions. (10 Marks)

1. **Parsing Strategy:** LL(1) is Top-Down; SLR, CLR, LALR are Bottom-Up Shift-Reduce.
2. **Grammar Power:** $\text{LL}(1) \subset \text{SLR}(1) \subset \text{LALR}(1) \subset \text{CLR}(1)$.
3. **State Count:** LL(1) uses non-terminal rows; LR(0)/SLR(1)/LALR(1) share identical K states; CLR(1) has $5 \times$ to $10 \times K$ states.
4. **Reduction Criteria:** SLR uses $\text{FOLLOW}(A)$; CLR uses specific lookahead item $[A \to \alpha\cdot, a]$; LALR merges lookaheads for identical cores.
5. **Implementation:** LL(1) uses recursive descent or $M[A, a]$ stack; LR parsers use deterministic shift-reduce finite state tables.

Q7. What is an Ambiguous Grammar? Prove that the expression grammar $E \to E + E \mid E * E \mid \mathbf{id}$ is ambiguous. (8 Marks)

A grammar is **Ambiguous** if there exists at least one input string with ≥ 2 distinct parse trees or leftmost derivations.
For string `id + id * id`: Derivation 1 applies $E \to E + E$ first (yielding $+ > *$), while Derivation 2 applies $E \to E * E$ first (yielding $* > +$). Because two distinct parse trees exist with differing precedence interpretations, the grammar is strictly ambiguous!

Q8. State the conditions under which a context-free grammar is guaranteed to be LL(1). (6 Marks)

A grammar G is LL(1) if and only if for every non-terminal A with distinct productions $A \to \alpha \mid \beta$:
1. $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$
2. If $\alpha \Rightarrow^* \epsilon$, then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$
3. If $\beta \Rightarrow^* \epsilon$, then $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$.

Q9. Explain the Closure and Goto functions used in constructing the canonical collection of LR items. (8 Marks)

• $\text{Closure}(\mathbf{I})$: If $[A \to \alpha \cdot B\beta] \in I$, add $[B \to \cdot\gamma]$ for every production $B \to \gamma$ in the grammar until no more items can be added.
• $\text{Goto}(\mathbf{I}, \mathbf{X})$: Defined as $\text{Closure}(\{[A \to \alpha X \cdot \beta] \mid [A \to \alpha \cdot X\beta] \in I\})$. Represents the target DFA state reached when grammar symbol X is shifted or transitioned on top of the parser stack.

Q10. Explain Error Recovery in Predictive LL(1) Parsers using Synchronizing Tokens and Panic Mode. (8 Marks)

When predictive parser table lookup $M[A, a]$ encounters an empty error entry:
1. **Synchronizing Tokens:** All terminal symbols in $\text{FOLLOW}(A)$ are designated as synchronizing tokens.
2. If token $a \in \text{FOLLOW}(A)$, the parser pops non-terminal A from the stack, assuming A has been satisfied, and resumes parsing.
3. If token $a \notin \text{FOLLOW}(A)$, the parser skips incoming token a from the input stream until a synchronizing token is reached.